Egyptian Chinese University
FACULTY OF ENGINEERING · MECHATRONICS

COMPUTER PROGRAMMING · FINAL PROJECT

3D Robotic Arm Simulator

A study of **Inverse Kinematics** and **linear path planning** for a two-link robotic arm in three-dimensional space.

COURSE
Computer Programming

SUPERVISOR
Dr. Reham Mohamed

TEAM LEADER
Mohammed Mohsen

SLIDE 01 · OVERVIEW

What did we build?

A program that takes a target point in 3D space and figures out the angles a 2-link robotic arm needs to reach it — then animates the motion smoothly between two points.

01 · INPUT

A target (X, Y, Z)

The user picks two points in space — a start point P1 and an end point P2 — using sliders.

02 · SOLVE

Calculate joint angles

The program computes three angles: **Base (θ0)**, **Shoulder (θ1)**, and **Elbow (θ2)** — using *Inverse Kinematics*.

03 · ANIMATE

Move smoothly

The arm moves from P1 to P2 step-by-step using **Linear Interpolation (LERP)**, recalculating angles at each step.

Our robotic arm

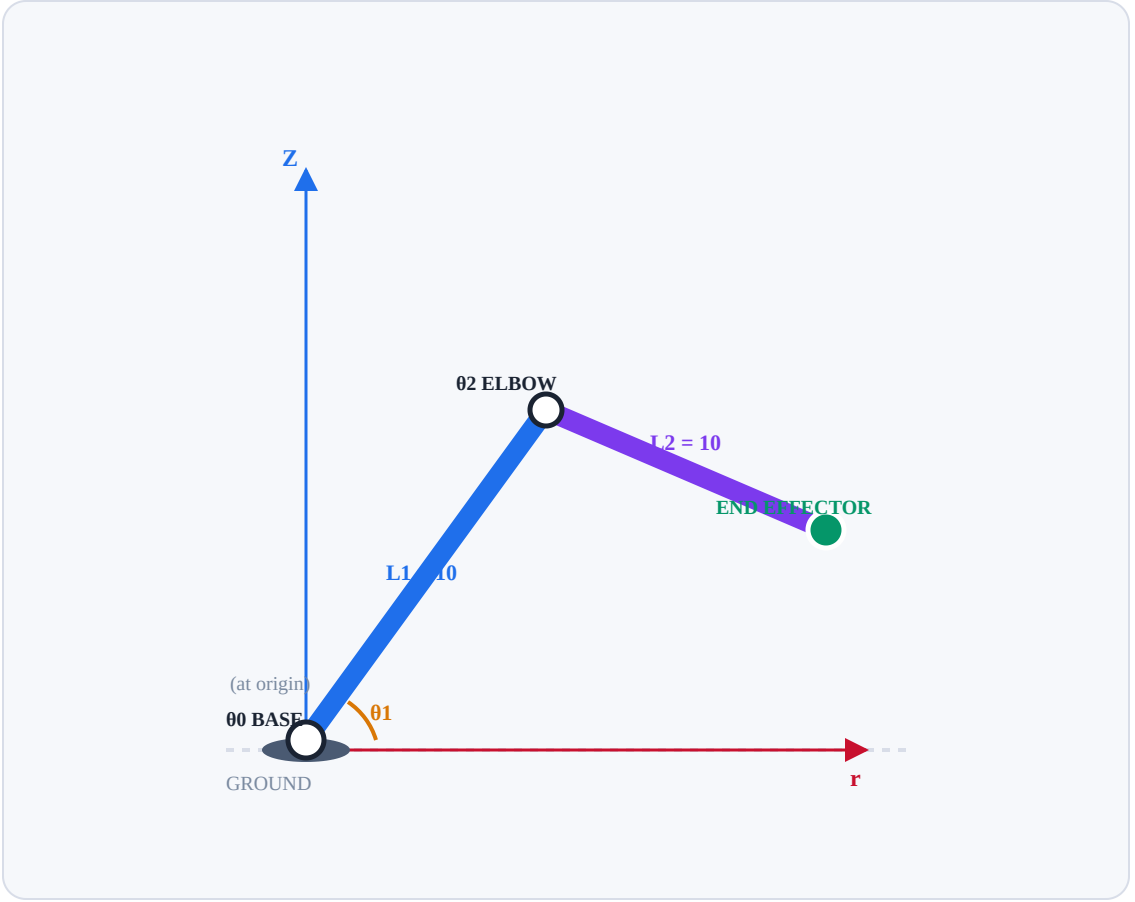
Two rigid links connected by joints. Both links are **10 units long** (L1 = L2 = 10).

1. **Base Joint (θ0)**
   Sits at the origin (0,0,0). Rotates the entire arm horizontally — left and right.

2. **Shoulder Joint (θ1)**
   Tilts Link 1 up or down — controls how high the arm reaches.

3. **Elbow Joint (θ2)**
   Bends Link 2 relative to Link 1 — controls how the arm folds.

4. **End Effector**
   The tip of Link 2 — this is what we want to position at the target point.



SLIDE 03 · THE BIG IDEA

Forward vs. Inverse Kinematics

There are two ways to think about a robotic arm — and we use the harder one.

FORWARD KINEMATICS · THE EASY ONE

Angles ➔ Position

You know the joint angles. You want to find where the end effector ends up.

Example: "If I rotate base by 30° and shoulder by 45°, where will the hand be?"

Just plug into trigonometry. Straightforward.

INVERSE KINEMATICS · THE HARD ONE

Position ➔ Angles

You know where you *want* the hand to go. You need to find what angles get you there.

Example: "I want to grab a cup at point (8, 6, 3). What angles should I set?"

Requires solving equations — this is what our project does.

WHY IT MATTERS IN MECHATRONICS

Real robots — surgical arms, factory assemblers, drone gimbals — all need IK to work. The user gives a target in space, and the controller must figure out the motor angles. Our simulator shows this principle in action.

The Base Angle (θ0)

Question: How far do we rotate the base — left or right — to face the target?

Answer: Use the X and Y coordinates of the target. The function `atan2(y, x)` gives us the correct angle in any quadrant — no sign mistakes.

FORMULA

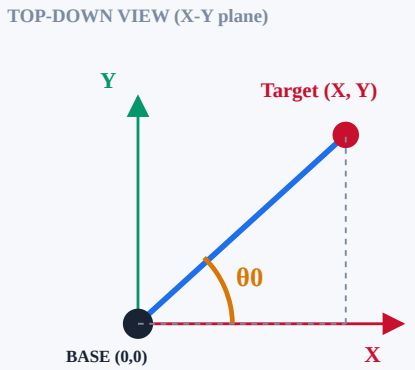
$$\theta_0 = \text{atan2}(Y, X)$$

EXAMPLE

Target = (X=10, Y=10, Z=5)

θ_0 = atan2(10, 10) = **45°**

The base swings 45° toward the target's horizontal direction.



From 3D to 2D — a simplification

After we handle the base rotation, the rest of the problem becomes 2D. The arm now moves in a vertical plane.

HORIZONTAL DISTANCE

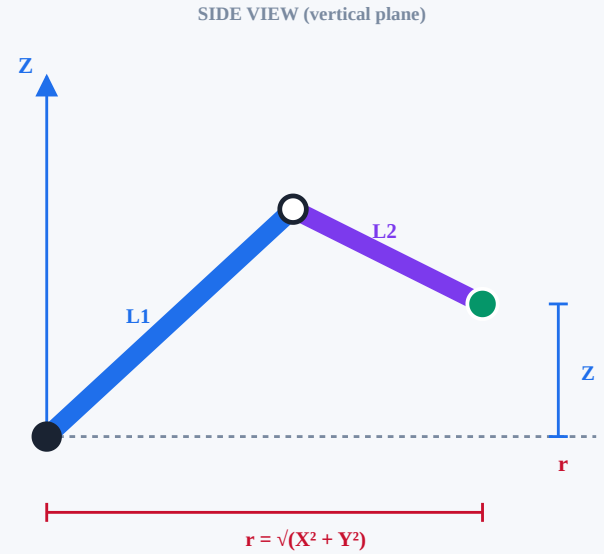
$$r = \sqrt{(X^2 + Y^2)}$$

We collapse X and Y into a single distance **r**. Combined with Z (the height), we now have a flat 2D problem to solve.

Example: Target (X=8, Y=6, Z=4)

r = √(64 + 36) = **10**

Now we solve in (r=10, z=4) — much simpler!



Solving Shoulder & Elbow Angles

Using the Law of Cosines on the triangle formed by Link 1, Link 2, and the line from base to target.

STEP A · HELPER VALUE D

$$D = (r^2 + Z^2 - L1^2 - L2^2) / (2 \cdot L1 \cdot L2)$$

STEP B · ELBOW ANGLE 02

$$\theta_2 = \pm \arccos(D)$$

STEP C · SHOULDER ANGLE 01

$$\theta_1 = \operatorname{atan2}(Z, r) - \operatorname{atan2}(L2 \cdot \sin\theta_2,\ L1 + L2 \cdot \cos\theta_2)$$

TWO SOLUTIONS: "ELBOW UP" & "ELBOW DOWN"

The ± sign in front of arccos(D) gives us **two valid configurations** — both reach the same point, but one bends the elbow upward and the other bends it downward. Our UI lets the user pick.

The IK function in C++

All the formulas above, translated into one function.

SOLVEIK( ) — CALCULATES JOINT ANGLES FOR A TARGET (X, Y, Z) C++

```
// Inputs: target position (x, y, z)
// Outputs: theta0 (base), theta1 (shoulder), theta2 (elbow)

double solveIK(double x, double y, double z) {
    double theta0 = atan2(y, x);             // base rotation

    double r    = sqrt(x*x + y*y);          // horizontal distance
    double dist = sqrt(r*r + z*z);          // total distance to target

    if (dist > L1 + L2)                    // safety check
        throw runtime_error("Out of reach!");

    double D = (r*r + z*z - L1*L1 - L2*L2) / (2 * L1 * L2);
    D = max(-1.0, min(1.0, D));         // clamp for safety

    double theta2 = acos(D);               // elbow angle (elbow-up)
    double theta1 = atan2(z, r)
                  - atan2(L2 * sin(theta2),
                          L1 + L2 * cos(theta2));
}
```

Moving smoothly: Linear Interpolation

To animate the arm from point P1 to point P2, we generate intermediate points along a straight line — and solve IK at each one.

LERP — LINEAR INTERPOLATION

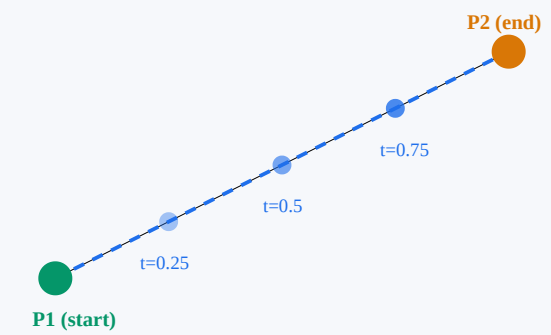
$$P(t) = P1 + t \cdot (P2 - P1)$$

where t ranges from 0 → 1

IN PLAIN ENGLISH

- **t = 0** → arm is at P1
- **t = 0.5** → arm is halfway
- **t = 1** → arm is at P2

With *num_steps* = *20*, we sample 20 evenly-spaced t values.



At each step we recalculate IK and update the arm.

Path generation in code

A simple loop that interpolates and calls the IK solver each step.

MOVELINEARLY( ) — ANIMATE FROM START TO END — C++

```cpp
void moveLinearly(double sx, double sy, double sz,    // start point P1
                  double ex, double ey, double ez,    // end point   P2
                  int num_steps) {

    for (int i = 0; i < num_steps; i++) {

        double t = (double)i / (num_steps - 1);   // 0 → 1

        // LERP — interpolate each axis
        double px = sx + t * (ex - sx);
        double py = sy + t * (ey - sy);
        double pz = sz + t * (ez - sz);

        // Solve IK for this intermediate point
        auto angles = solveIK(px, py, pz);

        // Move the arm and display angles on screen
        updateArm(angles);
        updateDisplay(px, py, pz, angles);
    }
}
```

"Out of Reach" — the safety check

The arm has a **maximum reach** — it cannot go beyond the sum of its links.

REACH CONSTRAINT

$$dist \leq L1 + L2 = 20 \text{ units}$$

If the user picks a point farther than 20 units from the base, the math becomes invalid (you can't take arccos of a number bigger than 1).

Our program **detects this**, stops the animation, and shows a warning toast on the UI instead of crashing.

⚠ WHY WE CLAMP D

Even when the target is reachable, floating-point rounding errors can push D slightly outside [-1, 1]. We clamp it to stay safe:

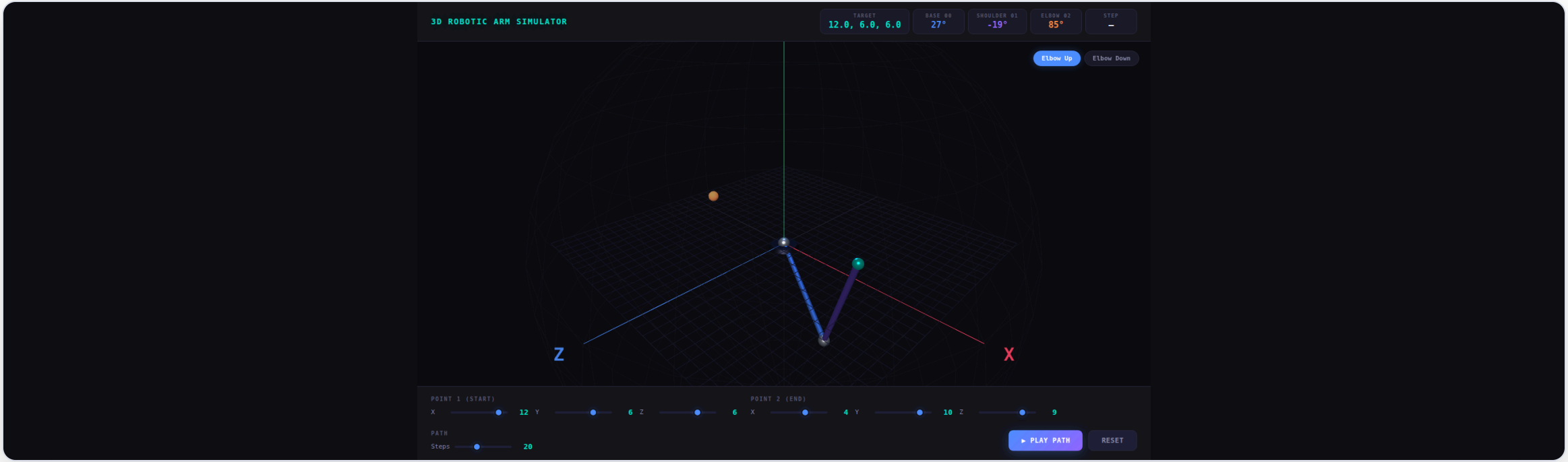
```
D = max(-1.0, min(1.0, D));
```

REACHABLE SPACE

All reachable points lie inside a **sphere of radius 20** centered at the base. We draw this sphere as a faint wireframe in the 3D view.

SLIDE 11 · THE INTERFACE

Our 3D Simulator UI



TOP **Header** shows live target position and joint angles.

MID **3D Canvas** renders the arm, axes, grid, and reach sphere.

SIDE **Toggle** switches between Elbow-Up and Elbow-Down solutions.

BTM **Controls** sliders set P1, P2, path steps + Play/Reset.

Current Point vs. Target Point

Every motion in our simulator is defined by two 3D points — the user controls both with sliders.

P1 · CURRENT POINT (GREEN MARKER)

Where the arm starts

This is the **initial position** of the end effector. When idle (not animating), the arm shows the pose for P1.

Sliders: P1 X, P1 Y, P1 Z — each ranges from −15 to +15.

P2 · TARGET POINT (ORANGE MARKER)

Where the arm should go

This is the **destination** the arm will move to when "Play Path" is pressed.

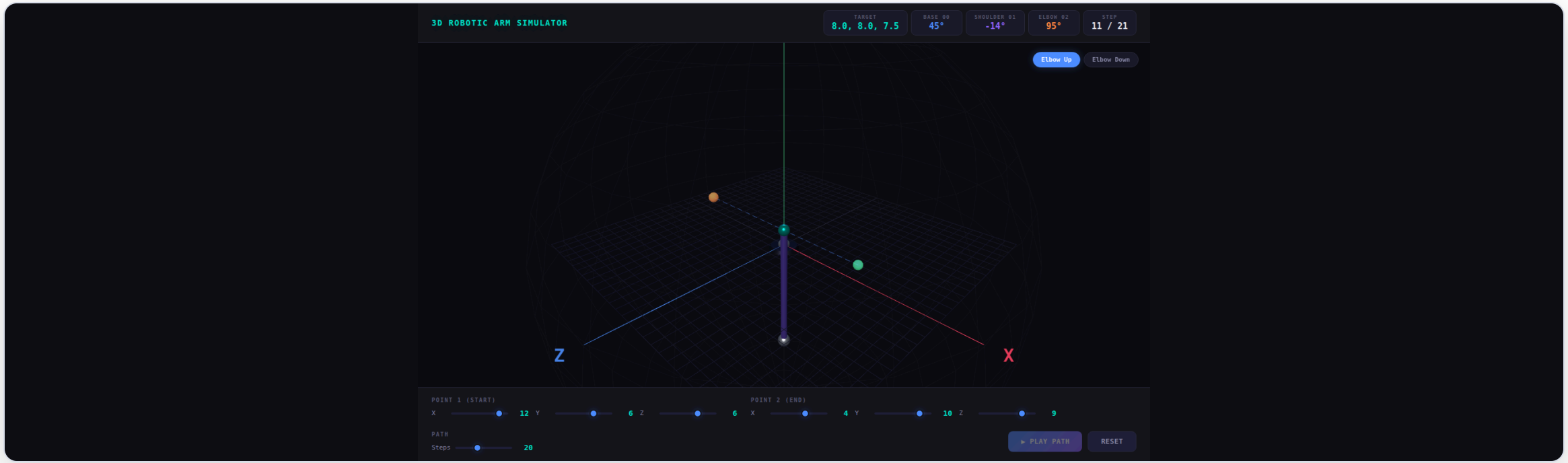
Sliders: P2 X, P2 Y, P2 Z — each ranges from −15 to +15.

IN THE 3D SCENE

- **Green sphere** = P1 (start) | **Orange sphere** = P2 (end) | **Cyan sphere** = current end effector position

Animation in action

A snapshot mid-motion — the dashed line is the planned path, the header updates angles in real time.



STEP **11 / 21** — animation is at frame 11 out of 21 total steps.

TARGET **(8.0, 8.0, 7.5)** — the LERP-interpolated point.

SHOULDER **−14°** — angle just recomputed by IK.

ELBOW **95°** — the bend at the elbow joint.

SLIDE 14 · THE TEAM

Project Team

First Year · Mechatronics Engineering · Egyptian Chinese University

TEAM LEADER

MM

Mohammed Mohsen
192400747

MA

Mostafa Ahmed Taha Mostafa
192500981

MN

Mahmoud Nasr Mahmoud
192400438

AM

Ahmed Mostafa Ebrahim
192400843

ZA

Ziad Arafa Salah
192400820

SUPERVISED BY

Dr. Reham Mohamed
Computer Programming

Thank You

Questions & Discussion